

L2 - 13.1. Basic Static Analysis

Task 1 - Introduction

Static analysis is the process of examining software code for bugs, security vulnerabilities, and code quality issues without executing the program. Learning Objectives:

- Lab setup for malware analysis
- Searching for strings in a malware
- Fingerprinting malware through hashes
- Signature-based detection mechanisms
- Extracting useful information from the PE header

Task 2 - Lab Setup

A resilient, isolated lab environment is essential when analyzing malware to safely contain its highly destructive risks.

FLARE VM: FLARE VM is a customizable, Windows-based virtual machine created by Mandiant that comes pre-equipped with tools for dynamic malware analysis.

REMnux: REMnux is a Linux-based malware analysis distribution pre-loaded with reverse engineering tools to streamline analysis workflows.

Task 3 - String search

A string search provides useful information to a malware analyst by identifying important pieces of strings present in a suspected malware sample.

How a string search works: A string search scans binary data for ASCII or Unicode character sequences ending in a null byte, which often produces false positive garbage alongside useful indicators that the analyst must filter.

When performing a string search, prioritize searching for high-value Indicators of Compromise (IOCs)—such as **Windows API functions** (which reveal functionality), **IP addresses or C2 domains** (which show network activity), and

contextual text like Bitcoin wallet addresses or ransom notes.

Both Linux and Windows feature a command-line `strings` utility—with Windows utilizing Microsoft's Sysinternals version—that extracts ASCII and Unicode text from binaries and can save the output to a file for analysis.

```
C:\Users\Administrator\Desktop>strings <path to binary>
```

String is a sequence of human-readable text characters embedded within computer data or programs.

Obfuscated strings are human-readable text sequences that malware intentionally encodes, encrypts, or hides to conceal its malicious intent from security analysis.

FLOSS (FLARE Obfuscated String Solver) is an automated malware analysis tool developed by Mandiant that extracts hidden and obfuscated strings from executable files.

```
C:\Users\Administrator\Desktop>floss --no-static-strings <path to binary>
```

 use FLOSS to search for obfuscated strings in a binary.

Answer the questions below

On the Desktop in the attached VM, there is a directory named 'mal' with malware samples 1 to 6. Use floss to identify obfuscated strings found in the samples named 2, 5, and 6. Which of these samples contains the string 'DbgView.exe'? 6

Open Command Prompt, navigate to the Desktop directory, and run the FLOSS commands individually. The query: C:\Users\Administrator\Desktop>floss --no-static-strings mal\6

Task 4 - Fingerprinting malware

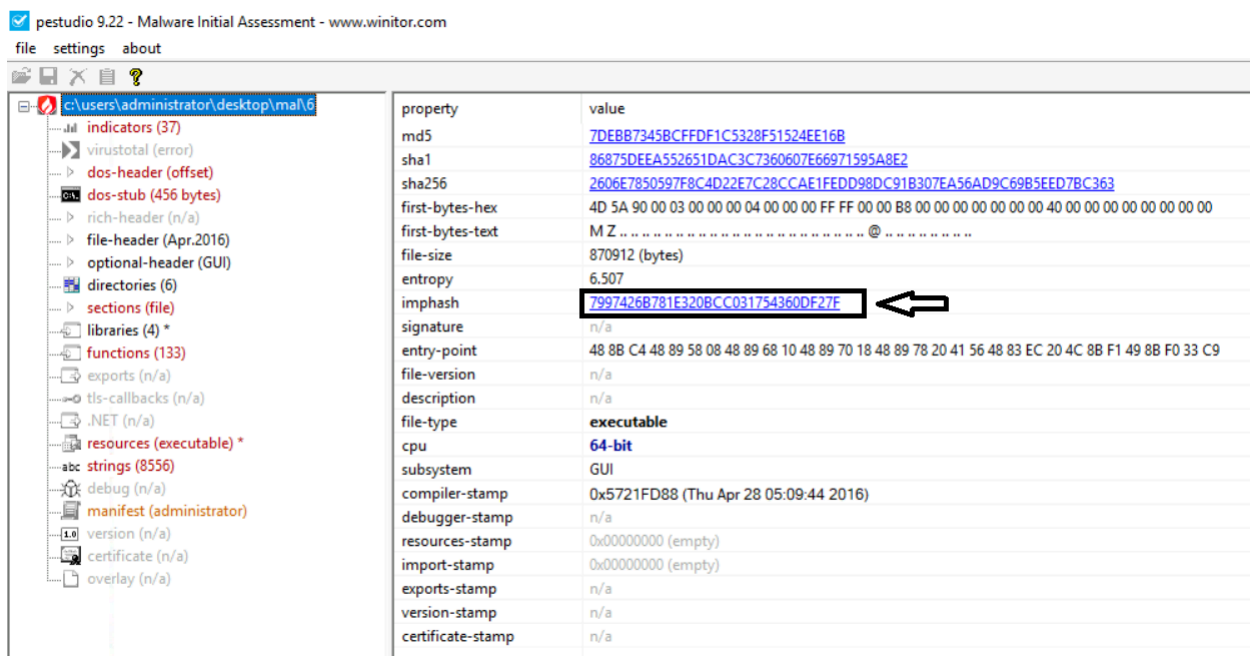
A hash function creates a unique, irreversible, fixed-length fingerprint based on a file's content—unaffected by file name changes—allowing analysts to reliably identify malware, search threat repositories, and use the hash as an Indicator of Compromise (IOC).

Commonly used methods of calculating File hashes:

- Md5sum
- Sha1sum
- Sha256sum

Finding Similar files using hashes:

Imphash: Imphash (Import Hash) is a unique cryptographic fingerprint generated from the specific DLLs, API functions, and import order used by an executable, allowing malware analysts to identify related samples, families, or threat groups even if the rest of the code change



Fuzzy hashes/SSDEEP: Fuzzy hashing (ssdeep) splits a file into smaller segments and hashes each piece individually, generating a Context Triggered Piecewise Hash (CTPH) that allows analysts to measure similarity percentages between two slightly modified malware files.

Answer the questions below

In the samples located at Desktop\mal\ directory in the attached VM, which of the samples has the same imphash as file 3? 1

Open all the files in PEStudio and check their Imphash values; 3rd matches the first one.

Using the ssdeep utility, what is the percentage match of the above-mentioned files? 93

C:\Users\Administrator\Desktop>ssdeep-2.14.1\ssdeep.exe -l -r -d mal (Run the query on CMD; 93 malware samples matched.) (ssdeep to recursively generate and compare fuzzy hashes for all files in the mal directory to detect matches or similarities.)

Task 5 - Signature-based detection

Signatures: Signatures are byte patterns inside a file that security researchers use to detect specific content, identify malicious behavior, and recognize malware families.

Yara rules: YARA rules are signature-based pattern-matching tools used to detect binary or textual sequences in files, though a match indicates a pattern rather than guaranteed malice.

Proprietary Signatures - AntiVirus Scans: Commercial antivirus signatures offer high-confidence threat detection with low false positives, though analysts should query file hashes on multi-scanner platforms like VirusTotal rather than uploading sensitive binaries.

Capa: Capa is an open-source Mandiant tool that scans executable artifacts—like imports, strings, and mutexes—to identify a binary's underlying functional capabilities.

```
C:\Users\Administrator\Desktop>capa mal.exe
```

Answer the questions below

How many matches for anti-VM execution techniques were identified in the sample? 86

Run `capa mal\4` ; after the report finishes generating, it shows that anti-VM execution techniques are mentioned 86 times.

Does the sample have to capability to suspend or resume a thread? Y

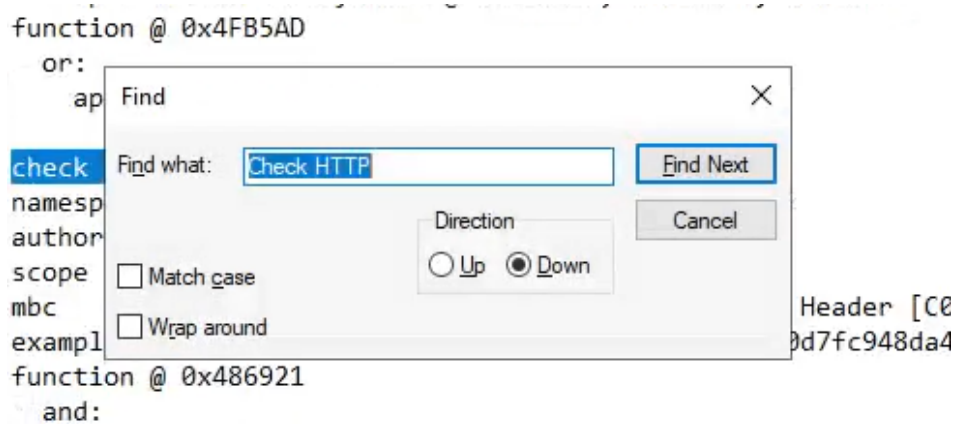
"Suspend or resume a thread" is listed as "Yes."

What MBC behavior is observed against the MBC Objective 'Anti-Static Analysis'?

Disassembler Evasion::Argument Obfuscation [B0012.001]

At what address is the function that has the capability 'Check HTTP Status Code'?

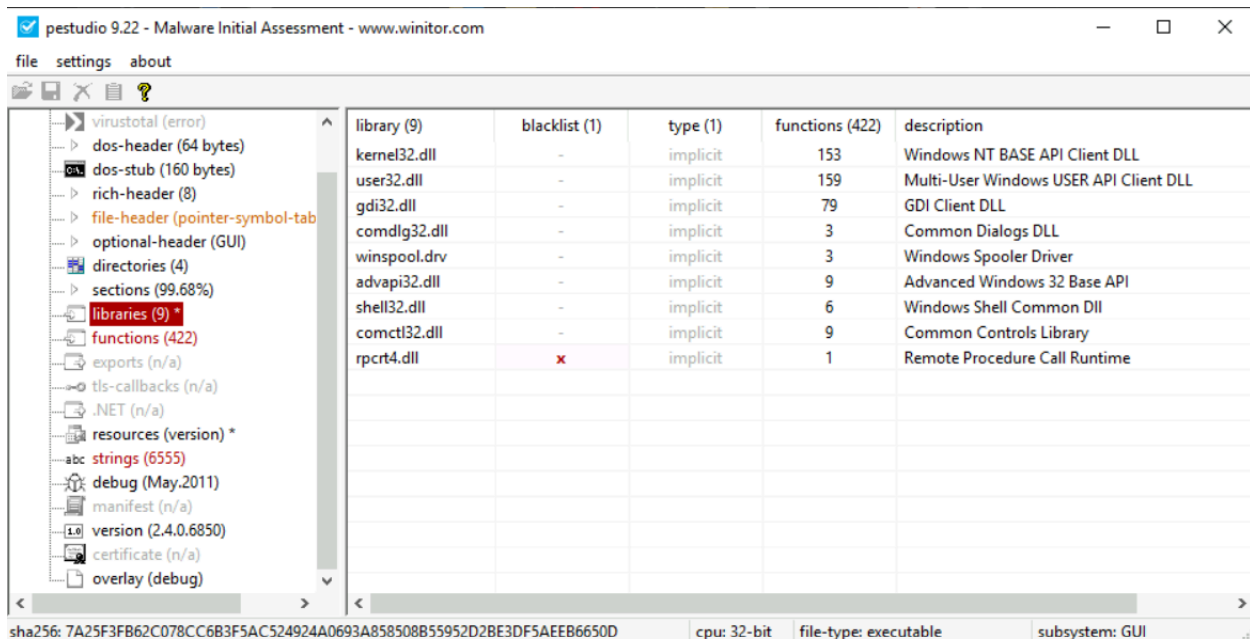
Run the command `capa -vv .\mal\4 > file.txt`, save the output to a file, and then filter for http.



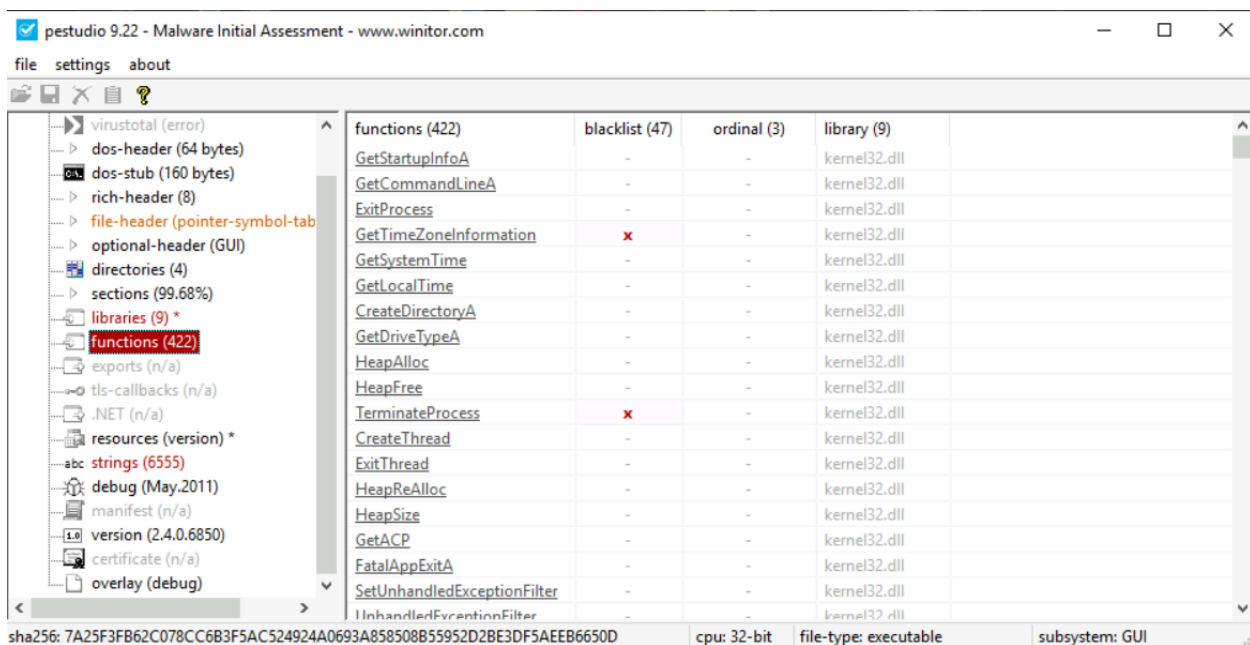
Task 6 - Leveraging the PE header

The PE header: The PE header acts as a foundational blueprint within a Portable Executable file, providing the operating system with the necessary characteristics and instructions to properly load and read the rest of the program's data.

Linked Libraries, imports, and functions: By examining the external libraries and specific functions a PE file imports from the Windows operating system, analysts can infer a malware sample's underlying capabilities and potential behavior before running it.



The functions option just below shows the functions imported from these libraries.

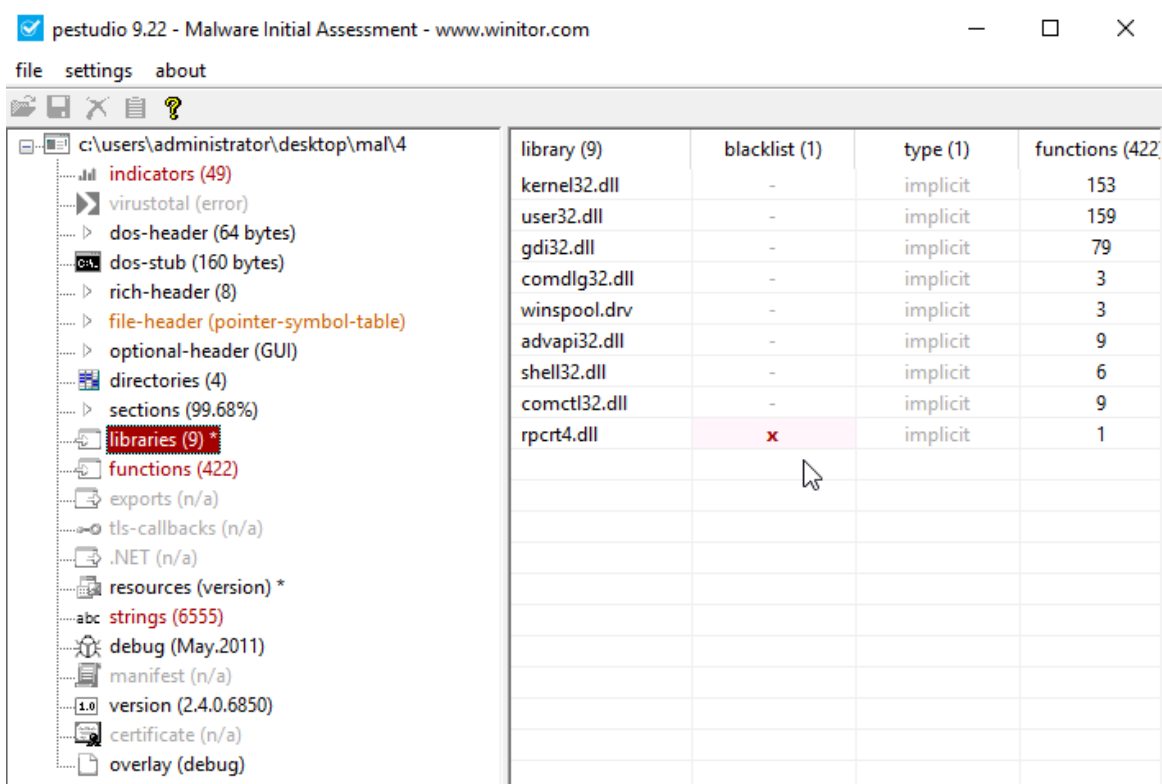


Identifying Packed Executables: Malware authors often encapsulate their code using packers to obfuscate its true properties and evade static analysis, but analysts can detect these wrappers by examining unusual section sizes and permissions within the PE header.

Answer the questions below

Open the sample Desktop\mal\4 in PEstudio. Which library is blacklisted?

run mal4 in pestudio



What does this dll do? read description

Task 7 - Conclusion

- Lab setup for malware analysis
- Searching for strings and obfuscated strings
- Fingerprinting malware using hashes and identifying similar samples using imphash and ssdeep
- Using signature-based detection like Yara and Capa
- Identifying artifacts from the PE header